Lecture 6: Model-Free Control

Rigorous Mathematical Notes

Based on lectures by Hado van Hasselt
UCL, 2021

Contents

1 Recap and Motivation

We work within the framework of a Markov Decision Process (MDP), possibly unknown.

Definition 1.1 (Markov Decision Process)**.** A *Markov Decision Process* is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \gamma)$, where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $p(s' \mid s, a)$ is the transition probability function, $r(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the expected reward function, and $\gamma \in [0,1)$ is the discount factor.

Definition 1.2 (Policy)**.** A *policy* π is a mapping from states to probability distributions over actions: $\pi(a \mid s) = \Pr(A_t = a \mid S_t = s)$.

Definition 1.3 (Value Functions)**.** The *state-value function* under policy π is

$$v_\pi(s) \;=\; \mathbb{E}_\pi\Big[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\Big|\; S_t = s\Big].$$

The *action-value function* under policy π is

$$q_\pi(s,a) \;=\; \mathbb{E}_\pi\Big[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\Big|\; S_t = s,\, A_t = a\Big].$$

Remark 1.1. In the previous lecture (Lecture 5), the focus was on *model-free prediction*: estimating v_π for a given policy π without knowledge of the MDP dynamics. In this lecture, we address *model-free control*: finding an optimal policy π^* that maximises the value function, again without knowledge of the MDP dynamics.

1.1 Recap: Model-Free Policy Evaluation

The general incremental update rule for state-value estimation is

$$v_{n+1}(S_t) \;=\; v_n(S_t) + \alpha\big(G_t - v_n(S_t)\big), \tag{1}$$

where G_t is a *return target*. Different choices of G_t yield different algorithms.

Definition 1.4 (Return Targets)**.** Let v_t denote the current value estimate at time t. The following targets define distinct model-free prediction algorithms:

(i) **Monte-Carlo (MC):**

$$G_t^{\mathrm{MC}} \;=\; R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \;=\; R_{t+1} + \gamma\, G_{t+1}^{\mathrm{MC}}.$$

(ii) **TD(0) (Temporal Difference):**

$$G_t^{(1)} \;=\; R_{t+1} + \gamma\, v_t(S_{t+1}).$$

(iii) **n-step TD:**

$$G_t^{(n)} \;=\; R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n v_t(S_{t+n}) \;=\; R_{t+1} + \gamma\, G_{t+1}^{(n-1)}.$$

(iv) **TD(λ):**

$$G_t^{\lambda} \;=\; R_{t+1} + \gamma\big[(1-\lambda)\, v_t(S_{t+1}) + \lambda\, G_{t+1}^{\lambda}\big].$$

In all cases, data is generated according to π, and the goal is estimating v_π.

2 Monte-Carlo Control

2.1 Generalised Policy Iteration (Refresher)

Definition 2.1 (Generalised Policy Iteration (GPI))**.** *Generalised Policy Iteration* refers to any scheme that alternates between two steps:

(i) **Policy Evaluation:** Estimate $v_\pi(s)$ (or $q_\pi(s,a)$) for the current policy π.

(ii) **Policy Improvement:** Generate a new policy π' such that $v_{\pi'}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$.

Repeated application converges to the optimal value function v^* and optimal policy π^*.

2.2 Model-Free Policy Iteration Using Action-Value Functions

Proposition 2.1 (Greedy Improvement over State Values Requires a Model)**.** *Given a state-value function $v(s)$, the greedy improved policy is*

$$\pi'(s) \;=\; \operatorname*{argmax}_{a\in\mathcal{A}} \mathbb{E}\big[R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t = s,\, A_t = a\big].$$

Computing this expectation requires knowledge of the transition dynamics $p(s' \mid s,a)$ and the reward function $r(s,a)$, i.e., a model of the MDP.

Proposition 2.2 (Greedy Improvement over Action Values is Model-Free)**.** *Given an action-value function $q(s,a)$, the greedy improved policy is*

$$\pi'(s) \;=\; \operatorname*{argmax}_{a\in\mathcal{A}} q(s,a).$$

This requires no knowledge of the transition dynamics or reward function.

Remark 2.1. This observation motivates the use of action-value functions $q(s,a)$ rather than state-value functions $v(s)$ for model-free control.

2.3 The Exploration Problem

Remark 2.2 (Need for Exploration)**.** In model-free GPI with action values, policy evaluation uses Monte-Carlo estimation of $q_\pi(s,a)$, which requires sampling all state-action pairs. If the policy improvement step uses a purely greedy policy, many state-action pairs will never be visited, preventing accurate estimation. This is the *exploration problem*.

Definition 2.2 (ε-Greedy Policy)**.** Given an action-value function $q(s,a)$ and a parameter $\varepsilon \in (0,1]$, the ε-greedy policy is defined as

$$\pi_\varepsilon(a \mid s) \;=\; \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}|} & \text{if } a = \operatorname{argmax}_{a'} q(s,a'), \\[2ex] \dfrac{\varepsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases}$$

2.4 Monte-Carlo Generalised Policy Iteration

The Monte-Carlo GPI algorithm alternates, after each episode, between:

(i) **Policy Evaluation:** Monte-Carlo estimation of $q \approx q_\pi$.

(ii) **Policy Improvement:** ε-greedy policy improvement.

Algorithm 1 Monte-Carlo Control with ε-Greedy Exploration

Initialise $q(s,a)$ arbitrarily for all $s \in \mathcal{S}$, $a \in \mathcal{A}$
$k \leftarrow 0$
repeat
  $k \leftarrow k+1$
  Sample episode $\{S_1, A_1, R_2, S_2, A_2, R_3, \ldots, S_T\}$ using $\pi = \varepsilon$-greedy(q)
  for each (S_t, A_t) appearing in the episode **do**
    Compute return $G_t = \sum_{i=0}^{T-t-1} \gamma^i R_{t+i+1}$
    $q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha_t\big(G_t - q(S_t, A_t)\big)$
  end for
  $\varepsilon \leftarrow 1/k$
  $\pi \leftarrow \varepsilon$-greedy$(q)$
until convergence

Remark 2.3. Typical choices for the step size include $\alpha_t = 1/N(S_t, A_t)$, where $N(S_t, A_t)$ is the visit count for pair (S_t, A_t), or $\alpha_t = 1/k$ where k is the episode number.

2.5 GLIE and Convergence of Monte-Carlo Control

Definition 2.3 (Greedy in the Limit with Infinite Exploration (GLIE))**.** A sequence of policies $\{\pi_t\}_{t=1}^{\infty}$ is said to be *Greedy in the Limit with Infinite Exploration* (GLIE) if the following two conditions hold:

(i) **Infinite exploration:** All state-action pairs are explored infinitely many times:

$$\forall\, s \in \mathcal{S},\ \forall\, a \in \mathcal{A}: \quad \lim_{t\to\infty} N_t(s,a) = \infty.$$

(ii) **Greedy convergence:** The policy converges to the greedy policy:

$$\lim_{t\to\infty} \pi_t(a \mid s) \;=\; \mathbb{1}\Big(a = \operatorname*{argmax}_{a'} q_t(s,a')\Big).$$

Example 2.1. The ε-greedy policy with $\varepsilon_k = 1/k$ satisfies the GLIE conditions: $\varepsilon_k \to 0$ ensures greedy convergence, while $\sum_k \varepsilon_k = \infty$ ensures infinite exploration.

Theorem 2.1 (Convergence of GLIE Monte-Carlo Control)**.** *If the sequence of policies is GLIE, then the Monte-Carlo control algorithm (Algorithm 1) converges to the optimal action-value function:*

$$q_t(s,a) \;\longrightarrow\; q_*(s,a) \quad \text{as } t \to \infty, \quad \forall\, s \in \mathcal{S},\ \forall\, a \in \mathcal{A}.$$

3 Temporal-Difference Learning for Control

3.1 Motivation: MC vs. TD Control

Remark 3.1 (Advantages of TD over MC for Control)**.** Temporal-difference learning offers several advantages over Monte-Carlo methods in the control setting:

(i) **Lower variance:** TD targets have lower variance than full Monte-Carlo returns.

(ii) **Online learning:** TD methods can update after every time step, without waiting for episode termination.

(iii) **Incomplete sequences:** TD methods can learn from incomplete episodes (e.g., in continuing tasks).

The natural idea is to apply TD learning to the action-value function $q(s,a)$, use ε-greedy policy improvement, and update at every time step.

3.2 SARSA: On-Policy TD Control

Definition 3.1 (SARSA Update)**.** The *SARSA* algorithm updates the action-value function using the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$:

$$q_{t+1}(S_t, A_t) \;=\; q_t(S_t, A_t) + \alpha_t\big(R_{t+1} + \gamma\, q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t)\big), \tag{2}$$

where A_{t+1} is chosen according to the current policy (e.g., ε-greedy with respect to q_t).

Remark 3.2. The name *SARSA* derives from the quintuple (S, A, R, S', A') used in each update.

SARSA performs GPI at every time step:

(i) **Policy Evaluation:** SARSA update (2), so that $q \approx q_\pi$.

(ii) **Policy Improvement:** ε-greedy policy improvement.

Algorithm 2 Tabular SARSA

Initialise $Q(s,a)$ arbitrarily for all $s \in \mathcal{S}$, $a \in \mathcal{A}$
repeat (for each episode)
  Initialise state s
  Choose a from s using policy derived from Q (e.g., ε-greedy)
  repeat (for each step of episode)
    Take action a, observe reward r and next state s'
    Choose a' from s' using policy derived from Q (e.g., ε-greedy)
    $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma\, Q(s',a') - Q(s,a)\big]$
    $s \leftarrow s';\;\; a \leftarrow a'$
  until s is terminal
until convergence

Theorem 3.1 (Convergence of Tabular SARSA)**.** *Tabular SARSA converges to the optimal action-value function,*

$$q(s,a) \;\longrightarrow\; q_*(s,a) \quad \text{as } t \to \infty,$$

provided the policy is GLIE (Definition 2.3) and the step sizes satisfy the Robbins–Monro conditions:

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

4 Off-Policy TD and Q-Learning

4.1 Dynamic Programming Background

Remark 4.1 (DP Algorithms and Their TD Analogues)**.** Several dynamic programming algorithms have direct model-free TD counterparts. The key DP algorithms for action values are:

$$q_{k+1}(s,a) = \mathbb{E}\big[R_{t+1} + \gamma\, q_k(S_{t+1}, A_{t+1}) \mid S_t = s,\, A_t = a\big] \qquad \text{(Policy Evaluation)}, \tag{3}$$

$$q_{k+1}(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} q_k(S_{t+1}, a') \mid S_t = s,\, A_t = a\Big] \qquad \text{(Value Iteration)}. \tag{4}$$

The model-free TD analogues are:

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t\big(R_{t+1} + \gamma\, q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t)\big) \qquad \textbf{(SARSA)}, \tag{5}$$

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t\Big(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t)\Big) \qquad \textbf{(Q-learning)}. \tag{6}$$

Remark 4.2 (No Trivial Analogue of State-Value Iteration)**.** There is no straightforward model-free sampling analogue of state-value iteration

$$v_{k+1}(s) = \max_a \mathbb{E}\big[R_{t+1} + \gamma\, v_k(S_{t+1}) \mid S_t = s,\, A_t = a\big],$$

because the max over actions is *outside* the expectation, and sampling provides an estimate conditioned on a particular action. One cannot take the max over actions from a single sample of S_{t+1}, since each action would lead to a different distribution over S_{t+1}. This is precisely why action-value formulations are essential for model-free control.

4.2 On-Policy vs. Off-Policy Learning

Definition 4.1 (On-Policy Learning)**.** In *on-policy* learning, the agent learns about the *behaviour policy* π from experience sampled from π itself. The policy being evaluated and the policy generating behaviour are the same.

Definition 4.2 (Off-Policy Learning)**.** In *off-policy* learning, the agent learns about a *target policy* π from experience sampled from a different *behaviour policy* μ. Formally, the goal is to estimate v_π or q_π using trajectories generated by μ, where in general $\pi \neq \mu$.

Remark 4.3 (Motivations for Off-Policy Learning)**.** Off-policy learning is important for several reasons:

(i) Learning from observing other agents or from logged data.

(ii) Reusing experience generated by old (previous) policies.

(iii) Learning about multiple target policies while following a single behaviour policy.

(iv) Learning about the greedy (optimal) policy while following an exploratory policy.

4.3 Q-Learning

Definition 4.3 (Q-Learning Update)**.** *Q-learning* updates the action-value function as follows:

$$q_{t+1}(S_t, A_t) \;=\; q_t(S_t, A_t) + \alpha_t\Big(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t)\Big). \tag{7}$$

The target policy is implicitly the greedy policy with respect to the current q, while the behaviour policy can be any sufficiently exploratory policy (e.g., ε-greedy).

Remark 4.4. Q-learning is an off-policy algorithm: it estimates the value of the greedy policy $\pi(s) = \operatorname{argmax}_a q(s,a)$, regardless of the behaviour policy used to generate the data. Acting greedily all the time would not explore sufficiently, so a separate exploratory behaviour policy is needed.

Theorem 4.1 (Convergence of Q-Learning)**.** *Q-learning converges to the optimal action-value function,*

$$q(s,a) \;\longrightarrow\; q^*(s,a) \quad \text{as } t \to \infty, \quad \forall\, s \in \mathcal{S},\ \forall\, a \in \mathcal{A},$$

provided:

(i) Each state-action pair (s,a) is visited infinitely often.

(ii) The step sizes satisfy the Robbins–Monro conditions:

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

In particular, $\alpha_t = 1/t^\omega$ with $\omega \in (0.5, 1)$ satisfies these conditions. Notably, no GLIE condition on the behaviour policy is required—any policy that selects all actions in all states sufficiently often suffices.

4.4 Cliff Walking Example

Example 4.1 (Cliff Walking)**.** Consider a gridworld where the agent starts at position S and must reach goal G along the bottom row. The cells between S and G on the bottom row constitute a "cliff": stepping into any cliff cell yields a reward of $r = -100$ and returns the agent to S. All other transitions yield $r = -1$.

With ε-greedy exploration (fixed ε):

- **SARSA** (on-policy) learns the "safe path" along the top of the grid, avoiding the cliff edge, because its value estimates account for the exploratory actions that could cause the agent to fall off the cliff.
- **Q-learning** (off-policy) learns the "optimal path" along the cliff edge (shortest path), because it evaluates the greedy policy. However, during training the agent occasionally falls off the cliff due to ε-greedy exploration, resulting in lower online reward per episode.

SARSA achieves higher online reward per episode, while Q-learning identifies the optimal deterministic policy.

5 Overestimation in Q-Learning

5.1 The Maximisation Bias

Proposition 5.1 (Overestimation Bias in Q-Learning)**.** *In Q-learning, the bootstrap target uses the same value function for both selecting and evaluating the best action:*

$$\max_a q_t(S_{t+1}, a) \;=\; q_t\Big(S_{t+1},\, \operatorname*{argmax}_a q_t(S_{t+1}, a)\Big).$$

Since the estimates q_t are noisy (approximate), the argmax *is more likely to select actions whose values are* overestimated *and less likely to select actions whose values are* underestimated*. This introduces a systematic upward (positive) bias:*

$$\mathbb{E}\Big[\max_a q_t(S_{t+1}, a)\Big] \;\geq\; \max_a \mathbb{E}\big[q_t(S_{t+1}, a)\big] \;=\; \max_a q_*(S_{t+1}, a),$$

where the inequality follows from Jensen's inequality applied to the convex function max*.*

Example 5.1 (Roulette)**.** Consider a single-state MDP with 171 actions: bet \$1 on one of 170 options (each with high variance and negative expected reward), or "stop" (which ends the episode with \$0 reward). The optimal policy is to stop immediately. However, Q-learning persistently overestimates the value of betting actions because, with 170 noisy estimates, the maximum is substantially biased upward. The estimated expected profit remains around \$20–\$40 above the true value of \$0 even after millions of updates.

5.2 Double Q-Learning

Definition 5.1 (Double Q-Learning)**.** *Double Q-learning* maintains two independent action-value functions q and q'. At each time step t, one of the two is selected (e.g., uniformly at random) for updating:

- If q is selected for updating, use q' to evaluate the action chosen by q:

$$q_{t+1}(S_t, A_t) \;=\; q_t(S_t, A_t) + \alpha_t\Big(R_{t+1} + \gamma\, q'_t\big(S_{t+1},\, \operatorname*{argmax}_a q_t(S_{t+1}, a)\big) - q_t(S_t, A_t)\Big). \quad (8)$$

- If q' is selected for updating, use q to evaluate the action chosen by q':

$$q'_{t+1}(S_t, A_t) \;=\; q'_t(S_t, A_t) + \alpha_t\Big(R_{t+1} + \gamma\, q_t\big(S_{t+1}, \operatorname*{argmax}_a q'_t(S_{t+1}, a)\big) - q'_t(S_t, A_t)\Big). \quad (9)$$

For action selection (behaviour), both estimates can be combined, e.g., using the ε-greedy policy with respect to $(q + q')/2$.

Remark 5.1 (Key Insight)**.** The critical idea is to *decouple selection from evaluation.* One value function determines which action is best (argmax), while an independent value function evaluates that action. Since the two are estimated from different samples, the positive correlation that causes overestimation in standard Q-learning is broken.

Theorem 5.1 (Convergence of Double Q-Learning)**.** *Double Q-learning converges to the optimal action-value function under the same conditions as standard Q-learning (Theorem 4.1): each state-action pair must be visited infinitely often, and the step sizes must satisfy the Robbins–Monro conditions.*

Remark 5.2 (Double DQN)**.** The Double Q-learning idea extends naturally to deep reinforcement learning. *Double DQN* uses the online network to select actions and a separate target network to evaluate them, substantially reducing overestimation in Atari game benchmarks compared to standard DQN.

5.3 Generalisation of Double Learning

Remark 5.3 (Double Learning Beyond Q-Learning)**.** The double learning idea generalises beyond Q-learning. Any algorithm that uses a (soft-)greedy policy—including SARSA with ε-greedy exploration—can exhibit overestimation bias due to the correlation between the policy and the value estimates. The same decoupling solution applies, yielding, e.g., *Double SARSA*.

6 Off-Policy Learning: Importance Sampling Corrections

6.1 Importance Sampling: General Framework

Definition 6.1 (Importance Sampling)**.** Let $f : \mathcal{X} \to \mathbb{R}$ be a function, d a target distribution over $\mathcal{X}$, and d' a proposal (behaviour) distribution over $\mathcal{X}$ with $d'(x) > 0$ whenever $d(x) > 0$. Then

$$\mathbb{E}_{x\sim d}\big[f(x)\big] \;=\; \sum_x d(x)\, f(x) \;=\; \sum_x d'(x)\, \frac{d(x)}{d'(x)}\, f(x) \;=\; \mathbb{E}_{x\sim d'}\left[\frac{d(x)}{d'(x)}\, f(x)\right]. \quad (10)$$

The ratio $\rho(x) = d(x)/d'(x)$ is called the *importance sampling ratio.*

Remark 6.1 (Intuition)**.** Importance sampling reweights samples from d' to correct for the distributional mismatch with d. Events that are rare under d' but common under d receive higher weight, and vice versa.

6.2 Importance Sampling for Reward Estimation

Proposition 6.1 (One-Step Importance Sampling Correction)**.** *Let μ be the behaviour policy and π the target policy. The expected one-step reward under π can be estimated from samples*

under μ:

$$\begin{aligned}\mathbb{E}\big[R_{t+1} \mid S_t = s,\, A_t \sim \pi\big] &= \sum_a \pi(a \mid s)\, r(s,a) \\ &= \sum_a \mu(a \mid s)\, \frac{\pi(a \mid s)}{\mu(a \mid s)}\, r(s,a) \\ &= \mathbb{E}\left[\frac{\pi(A_t \mid S_t)}{\mu(A_t \mid S_t)}\, R_{t+1} \;\middle|\; S_t = s,\, A_t \sim \mu\right]. \end{aligned} \tag{11}$$

Thus $\frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}\, R_{t+1}$ is an unbiased estimator of $\mathbb{E}_\pi[R_{t+1} \mid S_t = s]$ under behaviour μ.

6.3 Importance Sampling for Off-Policy Monte-Carlo

Proposition 6.2 (Monte-Carlo Importance Sampling). *Let $\tau_t = \{S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \ldots\}$ be a trajectory generated under behaviour policy μ. The importance sampling ratio for the full trajectory is:*

$$\begin{aligned}\frac{p(\tau_t \mid \pi)}{p(\tau_t \mid \mu)} &= \frac{\prod_{k=t}^{T-1} \pi(A_k \mid S_k)\, p(R_{k+1}, S_{k+1} \mid S_k, A_k)}{\prod_{k=t}^{T-1} \mu(A_k \mid S_k)\, p(R_{k+1}, S_{k+1} \mid S_k, A_k)} \\ &= \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}, \end{aligned} \tag{12}$$

where the transition probabilities $p(R_{k+1}, S_{k+1} \mid S_k, A_k)$ cancel. The importance-sampling corrected return is

$$\left(\prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}\right) G_t,$$

which provides an unbiased estimate of $v_\pi(S_t)$. Note that while unbiased, this estimator can have very high variance due to the product of ratios.

6.4 Importance Sampling for Off-Policy TD

Proposition 6.3 (Off-Policy TD with Importance Sampling). *For off-policy TD learning of v_π using data from μ, we weight the TD target by a single importance sampling ratio:*

$$v(S_t) \;\leftarrow\; v(S_t) + \alpha \left(\frac{\pi(A_t \mid S_t)}{\mu(A_t \mid S_t)}\big(R_{t+1} + \gamma\, v(S_{t+1})\big) - v(S_t)\right). \tag{13}$$

Only a single importance sampling correction is needed (not a product over the entire trajectory), resulting in much lower variance than Monte-Carlo importance sampling.

Proof. We verify that the expected update direction under μ equals the on-policy TD update direction under π:

$$\begin{aligned}&\mathbb{E}_\mu\left[\frac{\pi(A_t \mid S_t)}{\mu(A_t \mid S_t)}\big(R_{t+1} + \gamma\, v(S_{t+1})\big) - v(S_t) \;\middle|\; S_t = s\right] \\ &= \sum_a \mu(a \mid s)\left(\frac{\pi(a \mid s)}{\mu(a \mid s)}\, \mathbb{E}\big[R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t = s,\, A_t = a\big] - v(s)\right) \\ &= \sum_a \pi(a \mid s)\, \mathbb{E}\big[R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t = s,\, A_t = a\big] - \sum_a \mu(a \mid s)\, v(s) \\ &= \sum_a \pi(a \mid s)\, \mathbb{E}\big[R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t = s,\, A_t = a\big] - v(s) \\ &= \mathbb{E}_\pi\big[R_{t+1} + \gamma\, v(S_{t+1}) - v(s) \mid S_t = s\big], \end{aligned}$$

where we used $\sum_a \mu(a \mid s) = 1$ in the penultimate step. □

6.5 Expected SARSA

Definition 6.2 (Expected SARSA)**.** *Expected SARSA* updates the action-value function as:

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1})\, q(S_{t+1}, a) - q(S_t, A_t) \right), \tag{14}$$

where π is the target policy and the behaviour policy μ may differ from π. Crucially, no importance sampling correction is required because the expectation over the next action is computed exactly.

Remark 6.2. Expected SARSA subsumes Q-learning as a special case: when π is the greedy policy, we have

$$\sum_a \pi^{\text{greedy}}(a \mid S_{t+1})\, q(S_{t+1}, a) \;=\; \max_a q(S_{t+1}, a),$$

recovering the Q-learning update (7).

Remark 6.3 (No Importance Sampling Needed)**.** Expected SARSA computes an explicit expectation over the target policy's action probabilities, eliminating the need for importance sampling. The behaviour policy μ determines which state-action pairs are visited, but the update target itself is constructed using π.

7 Summary and Comparison of Algorithms

7.1 Unified View of Model-Free Control

Remark 7.1 (Model-Free Policy Iteration)**.** The overarching framework for model-free control is as follows:

(i) Learn action values $q(s, a)$ to predict the current (or target) policy π.

(ii) Perform policy improvement, e.g., make the policy greedy: $\pi \to \pi'$.

(iii) Repeat.

The algorithms discussed differ in their choice of target policy and update mechanism:

Definition 7.1 (Algorithm Classification)**.** (i) **SARSA** (On-policy): Uses a stochastic sample from the behaviour policy as the target:

$$\text{target} = R_{t+1} + \gamma\, q(S_{t+1}, A_{t+1}), \qquad A_{t+1} \sim \pi(\cdot \mid S_{t+1}).$$

Convergence to q^* requires π to satisfy the GLIE conditions (Definition 2.3).

(ii) **Q-Learning** (Off-policy): Uses a greedy target policy:

$$\text{target} = R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a').$$

Akin to value iteration; immediately estimates the optimal (greedy) policy.

(iii) **Expected SARSA** (Off-policy, general): Uses any target policy π:

$$\text{target} = R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1})\, q(S_{t+1}, a).$$

Subsumes Q-learning (when π is greedy) and on-policy Expected SARSA (when $\pi = \mu$).

(iv) **Double Learning**: Uses a separate value function to evaluate the policy selected by the primary value function. Applicable to any of the above algorithms.

7.2 When is Double Learning Useful?

Remark 7.2 (Necessity of Double Learning)**.** Double learning addresses the overestimation bias caused by using the same value function for both action selection and action evaluation.

(i) In *pure prediction* (policy evaluation for a fixed π), there is no correlation between the target policy and the value function estimates, so double learning is unnecessary.

(ii) When using a *greedy target policy* (Q-learning), the target policy is defined directly in terms of the value estimates, creating strong correlations. Double learning (Double Q-learning) is particularly beneficial in this setting.

(iii) When using a *soft-greedy policy* (ε-greedy SARSA), overestimation can also occur, and Double SARSA provides the analogous correction.

7.3 Off-Policy Control with Q-Learning: Detailed View

Remark 7.3 (Q-Learning as Off-Policy Expected SARSA)**.** In Q-learning, the target policy π is greedy with respect to q:

$$\pi(S_{t+1}) = \operatorname*{argmax}_{a'} q(S_{t+1}, a'),$$

while the behaviour policy μ can be any exploratory policy (e.g., ε-greedy w.r.t. q). The Q-learning target can be written as:

$$R_{t+1} + \gamma \sum_a \pi^{\text{greedy}}(a \mid S_{t+1})\, q(S_{t+1}, a) \;=\; R_{t+1} + \gamma \max_a q(S_{t+1}, a).$$

Both the behaviour and target policies improve as q improves.

7.4 On-Policy Control with SARSA: Detailed View

Remark 7.4 (SARSA Convergence Requirements)**.** In SARSA, the target and behaviour policies are identical. The target is simply:

$$\text{target} = R_{t+1} + \gamma\, q(S_{t+1}, A_{t+1}), \qquad A_{t+1} \sim \pi(\cdot \mid S_{t+1}).$$

For convergence to q^*, the additional requirement is that π eventually becomes greedy (e.g., via decreasing ε in ε-greedy, or decreasing temperature in softmax).

8 Convergence Conditions: Summary

Assumption 8.1 (Robbins–Monro Step-Size Conditions)**.** The step-size sequence $\{\alpha_t\}_{t\geq 1}$ satisfies:

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \text{and} \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

The following table summarises the convergence guarantees:

Algorithm	On/Off-Policy	Converges to q^*?	Conditions
MC Control	On-policy	Yes	GLIE
SARSA	On-policy	Yes	GLIE + Robbins–Monro
Q-Learning	Off-policy	Yes	Infinite visits + Robbins–Monro
Double Q-Learning	Off-policy	Yes	Same as Q-Learning
Expected SARSA	Off-policy	Yes	Depends on π